

HIGHER DIPLOMA IN SOFTWARE ENGINEERING (HDSE)

INSTITUTE OF SOFTWARE ENGINEERING (IJSE)

COURSEWORK PROJECT REPORT

ETech Computers

**Multi-Branch Computer Hardware E-Commerce
& Enterprise Management Platform**

Advanced API Development — ITS1114

HDSE 75

Academic Year 2026

Submission: September 2026

Prepared by: _____

Student Registration No.: _____

Document Version 1.0

Declaration of Originality

I hereby declare that this project report and the accompanying software implementation represent my own academic work, developed for the ITS1114 Advanced API Development module. All external technologies, frameworks, libraries and referenced technical materials used in the project are acknowledged appropriately. I understand and accept the academic integrity and plagiarism requirements of the Institute of Software Engineering.

Acknowledgement

I would like to express my sincere appreciation to the lecturers and academic staff of the Institute of Software Engineering for their guidance throughout the Higher Diploma in Software Engineering programme. I also acknowledge the contribution of the ETech Computers business context, which provided the operational requirements used to design this system. Finally, I appreciate the support of colleagues, friends and family during the analysis, implementation, testing and documentation stages of the project.

Abstract

ETech Computers is a multi-branch computer hardware retail and distribution platform designed to centralize product management, branch inventory, customer ordering, inter-branch stock transfers, promotions, customer engagement and administrative operations. The project addresses operational limitations associated with fragmented inventory information, manual stock coordination, inconsistent fulfilment processes and limited access control.

The solution adopts an API-first architecture with a Spring Boot REST backend, Spring Security, JSON Web Tokens, BCrypt password hashing, Spring Data JPA/Hibernate and a MySQL relational database. A lightweight Single Page Application implemented with Vanilla JavaScript and Tailwind CSS consumes the REST API. The database contains 24 normalized entities supporting catalog management, branch inventory, orders, reviews, transfers, promotions, bundles, newsletters and legal content.

The implementation also includes value-added capabilities such as an AI hardware advisor, branch analytics, Haversine-based fulfilment routing, composite bundle stock calculation, automated product badging, newsletter management and multi-stage stock transfers. Testing covers authentication, authorization, validation, inventory, ordering, logistics, transfers, AI interaction and analytics. The resulting system demonstrates the application of RESTful API design, secure backend development, relational database engineering and full-stack integration within an HDSE coursework context.

Table of Contents

This document is structured into ten principal chapters followed by references and appendices. In Microsoft Word, the Table of Contents can be updated automatically using References → Table of Contents.

List of Tables

Table 1 — Project objectives

Table 2 — Stakeholder roles

Table 3 — Functional requirements

Table 4 — Non-functional requirements

Table 5 — Technology stack

Table 6 — RBAC permissions

Table 7 — Database entities

Table 8 — API modules

Table 9 — Test execution matrix

Table 10 — Future enhancement roadmap

List of Figures

Figure 1 — Legacy operational problem model

Figure 2 — Proposed API-first architecture

Figure 3 — System use-case model

Figure 4 — High-level system architecture

Figure 5 — Security request flow

Figure 6 — Domain component interaction

Figure 7 — Customer order sequence

Figure 8 — Stock transfer sequence

Figure 9 — Database normalization flow

Figure 10 — Frontend routing flow

Abbreviations

Term	Meaning
------	---------

API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
ORM	Object-Relational Mapping
JPA	Java Persistence API
SPA	Single Page Application
SQL	Structured Query Language
JPQL	Jakarta Persistence Query Language
DTO	Data Transfer Object
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
NFR	Non-Functional Requirement
FR	Functional Requirement
LKR	Sri Lankan Rupees
ERD	Entity Relationship Diagram
LTS	Long-Term Support

1. Introduction

1.1 Background

The ETech Computers project was developed as an API-focused software engineering solution for a computer hardware retail environment. The platform combines an online storefront with administrative capabilities so that catalog information, branch inventory, customer orders and operational workflows can be managed through a centralized backend.

The original project specification identifies operational challenges associated with decentralized stock records, manual stock transfers, disconnected customer channels, inconsistent fulfilment routing and weak security controls. These problems create unnecessary communication overhead and make it difficult for staff and administrators to obtain a consistent view of current business information.

1.2 Organizational Context

The documented system models ETech Computers as a computer hardware business operating across regional hubs in Colombo, Galle, Matara and Kandy. The platform supports individual customers as well as organizational and enthusiast hardware requirements. Its product domain includes laptops, computer components, peripherals and related hardware categories.

Area	System representation
Branches	Colombo, Galle, Matara and Kandy regional hubs
Catalog	Products, brands, categories, badges, images, specifications and features
Inventory	Branch-level stock quantities and stock-health monitoring
Customers	Registration, orders, wishlists, reviews and tracking
Operations	Stock transfers and order fulfilment
Marketing	Deals, bundles, banners and newsletters
Administration	Users, roles, analytics, business profile and policies

1.3 Problem Statement

A multi-branch hardware retailer requires accurate and timely information about products, stock, customer orders and branch operations. When these activities are managed using disconnected records or manual communication, staff may not have a reliable view of stock availability, transfers can be difficult to reconcile, and customers may need to contact branches individually to confirm product information.

The project therefore addresses the following core problem: how to design and implement a secure, centralized and API-driven platform capable of supporting multi-branch inventory, online ordering, administrative operations and customer-facing services while maintaining data integrity and controlled access.

1.4 Proposed Solution

The proposed solution is a centralized RESTful backend that exposes business resources through JSON APIs. The frontend communicates with this backend through HTTP requests, while Spring Security validates authenticated requests. Business rules are handled in service classes and persistent data is managed through Spring Data JPA/Hibernate against MySQL.

1.5 Aim

The primary aim is to design and implement a secure, maintainable and extensible multi-branch computer hardware e-commerce and management platform that demonstrates advanced REST API development and relational database engineering.

1.6 Objectives

- Develop a resource-oriented REST API using Spring Boot and Java.
- Implement stateless authentication using JWT and secure password hashing.
- Implement role-based authorization for administrative, staff and customer functions.
- Design a normalized relational database supporting the required business domains.
- Provide centralized branch inventory and inter-branch stock transfer workflows.
- Support customer ordering, tracking, reviews and wishlists.
- Implement product filtering, pagination and advanced data retrieval queries.
- Provide analytics and value-added features such as AI assistance and dynamic fulfilment routing.
- Integrate the backend with a responsive Single Page Application.
- Verify the system through unit, integration and API-level testing.

1.7 Scope

In Scope	Description
Authentication	Login, registration, JWT validation and role authorization
Catalog	Product, category, brand, specifications, features and images
Inventory	Branch-level quantities, stock alerts and transfers
Orders	Order placement, line items, fulfilment status and tracking
Customer engagement	Wishlist, reviews and newsletter subscription

Promotions	Hot deals, banners and composite bundles
Administration	User management, analytics, business profile and policies
Advanced functions	AI advisor, Haversine routing and automated badging
Deployment	Docker-oriented backend deployment configuration

1.8 Limitations

The documented implementation focuses on the coursework scope. The future roadmap identifies payment gateway webhooks, SMS notifications, Elasticsearch-based search and a native mobile application as potential future extensions. These are therefore treated as future work rather than completed system capabilities.

1.9 Learning Outcome Alignment

Learning Outcome	Evidence in Project
LO1	REST controllers, resource-oriented endpoints, HTTP methods and standardized responses
LO2	JWT authentication, Spring Security, BCrypt, validation and RBAC
LO3	24-table normalized MySQL schema integrated through JPA/Hibernate
LO4	JPQL/SQL joins, filtering, pagination and aggregation queries

2. Requirements Engineering

2.1 Stakeholder Analysis

Stakeholder	Responsibilities / Needs
Super Administrator	System-level administration, users, policies, business configuration and high-level analytics
Administrator	Branch operations, approvals, promotions and staff management
Staff	Inventory operations, stock transfers, order fulfilment and alerts
Registered Customer	Catalog browsing, checkout, tracking, wishlist and reviews
Guest Visitor	Public catalog browsing, chatbot interaction and public order tracking

2.2 Functional Requirements

ID	Requirement	Summary
FR-01	Authentication & identity management	Authenticate users and issue signed JWT tokens.
FR-02	User administration	Manage user accounts, roles and statuses.
FR-03	Product catalog	Maintain products, technical specifications, features and galleries.
FR-04	Branch inventory	Maintain stock quantities by branch and generate low-stock alerts.
FR-05	Stock transfers	Create, approve, dispatch and complete inter-branch transfers.
FR-06	Customer orders	Create orders, validate stock and maintain fulfilment information.
FR-07	Promotions & bundles	Manage hot deals and calculate bundle availability.
FR-08	Taxonomy & badges	Maintain category hierarchy and product merchandising badges.
FR-09	Wishlist	Allow customers to save products and move saved items to cart.
FR-10	Reviews	Allow ratings and textual reviews for purchased

		products.
FR-11	Newsletter	Manage subscriptions and administrative campaigns.
FR-12	AI & analytics	Provide hardware advisory interactions and business reporting.

2.3 Non-Functional Requirements

ID	Quality Attribute	Implementation Approach
NFR-01	Performance	Target efficient response times through indexed queries, pagination and fetch optimization.
NFR-02	Security	Protect credentials and API resources using BCrypt, JWT and authorization rules.
NFR-03	Integrity	Maintain transactional consistency for orders and stock mutations.
NFR-04	Scalability	Use stateless authentication so backend instances do not depend on server sessions.
NFR-05	Reliability	Provide frontend fallback persistence for selected client-side data.
NFR-06	Maintainability	Use layered architecture, modular services and container-oriented deployment.

2.4 Use Cases

The major use cases are grouped around customer services, warehouse operations and administrative management. Public users can browse products and interact with the advisory service. Customers can manage personal shopping activities. Staff can manage inventory and fulfilment, while administrators control operational approvals and business configuration.

Use Case	Primary Actor	Outcome
Browse & filter products	Guest / Customer	Relevant catalog results displayed
Place order	Guest / Customer	Order created and stock reserved/decremented
Track order	Guest / Customer	Order status retrieved using tracking code

Manage inventory	Staff / Admin	Branch stock maintained
Transfer stock	Staff / Admin	Inventory rebalanced through controlled workflow
Manage products	Admin	Catalog information created or updated
View analytics	Admin	Sales and branch metrics displayed
Manage users	Admin / SuperAdmin	Accounts and roles maintained
Use AI advisor	Public users	Hardware guidance and product suggestions returned

2.5 Core Business Rules

- Product SKUs must be unique.
- Branch inventory is maintained separately for each product and branch combination.
- Order placement must validate sufficient branch stock before completing the transaction.
- Stock transfer completion changes source and destination inventory within a transactional operation.
- Protected endpoints require authentication and appropriate authorization.
- Customer reviews are associated with products and can identify verified purchases.
- Composite bundle availability is constrained by the least-available component.
- Only active catalog products should be exposed as purchasable products.

3. System Architecture and Design

3.1 Architectural Style

The system follows an N-Tier layered architecture. The presentation layer contains the browser-based SPA. The security tier processes CORS and JWT authentication. The application tier contains REST controllers, DTOs and business services. The persistence tier uses repositories and Hibernate ORM, while MySQL provides durable relational storage.

Layer	Main Components	Responsibility
Presentation	HTML, Vanilla JS, Tailwind CSS	UI rendering, routing and user interaction
Security	Spring Security, JWT filter	Authentication, authorization and request protection
API	REST Controllers, DTOs	HTTP request/response handling
Business	Service interfaces and implementations	Business rules and transactions
Persistence	Spring Data JPA repositories	Data access and query execution
ORM	Hibernate	Object-relational mapping
Database	MySQL 8.x	Persistent relational data

3.2 Technology Stack

Technology	Purpose
Java 21	Backend programming language
Spring Boot	REST API application framework
Spring Security	Authentication and authorization
JWT	JWT creation and validation
Spring Data JPA	Repository abstraction
Hibernate ORM	Object-relational mapping
MySQL 8.x	Relational database
Jakarta Validation	Request validation
Vanilla ES6+ JavaScript	SPA application logic
Tailwind CSS	Responsive UI styling
Docker	Containerized deployment
JUnit 5 / Mockito	Unit testing
MockMvc	Spring integration testing
Postman / cURL	API verification

3.3 Component Responsibilities

Controllers are responsible for HTTP-level concerns. Services encapsulate business operations and transactional boundaries. Repositories provide persistence operations. DTOs isolate API contracts from persistence entities. Entities model database relationships. This separation reduces coupling and makes individual layers easier to maintain and test.

3.4 Security Architecture

After successful authentication, the backend issues a signed JWT. Subsequent protected requests include the token in the Authorization header using the Bearer scheme. The JWT filter validates the token and establishes the authenticated principal in the Spring Security context. Method-level authorization can then enforce permissions through annotations such as `@PreAuthorize`.

Role	Typical Access
SUPERADMIN	Master system administration and unrestricted administrative capabilities
ADMIN	Branch administration, approvals, catalog and analytics
STAFF	Inventory, transfers and order fulfilment
CUSTOMER	Shopping, wishlist, reviews and personal order access
GUEST	Public catalog, chatbot and public tracking

3.5 Domain Interaction Example

A product-management request enters `ProductController`, is validated and passed to `ProductService`. The service resolves related category, brand and badge data and delegates persistence to `ProductRepository`. The repository uses Hibernate to translate the operation into SQL against MySQL. A standardized `CommonResponse` object is then returned to the client.

3.6 Order Processing Flow

1. Customer submits checkout information and selected items.
2. Frontend sends an order request to the REST API.
3. Order service validates the selected fulfilment branch.
4. Each requested product is checked against branch inventory.
5. Stock is decremented when sufficient inventory is available.
6. Order and order-item records are persisted within the transaction.
7. Subtotal, tax, shipping and total amounts are calculated.
8. A unique tracking code is returned to the client.
9. The frontend displays the order confirmation and tracking information.

3.7 Stock Transfer Flow

10. Staff creates a transfer request with source, destination, product and quantity.
11. Backend verifies source stock availability.
12. Transfer is created with a controlled workflow status.
13. Administrator approves or rejects the request.
14. Approved stock can progress through dispatch and completion.
15. Completion performs the source decrement and destination increment transactionally.
16. The completed transfer becomes part of the operational record.

4. Database Design

4.1 Database Design Methodology

The database is organized around domain-specific entities rather than storing all business information in large flat records. Relationships are enforced through foreign keys and uniqueness constraints. The design separates catalog data from branch stock, customer activity from orders, and promotion configuration from core products.

4.2 Entity Relationship Overview

The principal relationships include branches-to-users, branches-to-inventory, users-to-orders, products-to-categories and brands, products-to-images/specifications/features, orders-to-order-items, products-to-wishlists/reviews, and products-to-stock transfers. Composite business structures such as deal bundles are represented through associative bundle_items records.

4.3 Database Entity Catalogue

#	Entity	Purpose
1	branches	Regional hubs and fulfilment locations
2	users	System identities, credentials, roles and branch assignments
3	categories	Hierarchical product classification
4	brands	Hardware manufacturers
5	badges	Product merchandising labels and automation rules
6	products	Master hardware catalog
7	product_images	Product gallery assets
8	specs	Normalized technical specifications
9	features	Product feature bullet points
10	branch_inventory	Product stock per branch
11	orders	Customer order headers
12	order_items	Products contained in orders
13	wishlist_items	Customer saved products
14	product_reviews	Customer ratings and reviews
15	stock_transfers	Inter-branch inventory movement
16	hot_deals	Time-limited product promotions
17	home_deal_banners	Homepage promotional configurations
18	deal_bundles	Composite product bundles

19	bundle_items	Products required by each bundle
20	newsletter_subscribers	Newsletter subscription records
21	newsletter_campaigns	Marketing campaign records
22	business_profiles	Corporate profile configuration
23	legal_policies	Customer-facing policy records
24	policy_sections	Structured policy content

4.4 Selected Entity Data Dictionary

Entity	Important attributes
branches	id, name, city, address, phone, email, latitude, longitude, base_shipping_rate, active
users	id, name, username, email, password, phone, role, status, branch_id, timestamps
products	id, name, sku, category_id, brand_id, badge_id, price, original_price, rating, reviews_count, descriptions, warranty, stock-alert fields
branch_inventory	id, product_id, branch_id, stock_quantity; unique product/branch combination
orders	id, order_code, user_id, customer details, shipping data, fulfilment branch, distance, financial totals, status, payment method
order_items	id, order_id, product_id, historical product name/SKU, unit price, quantity, total
stock_transfers	id, transfer_number, source/destination branch, product, quantity, status, request/approval users, timestamps
deal_bundles	id, title, description, bundle price, discount, original price, active flag
bundle_items	id, bundle_id, product_id, quantity
product_reviews	id, product_id, user_id, author, rating, text, verified purchase, timestamp

4.5 Constraints and Integrity

- Unique constraints prevent duplicate usernames, emails, SKUs and newsletter subscriptions.
- Foreign keys enforce valid relationships between domain entities.
- ON DELETE behavior is used selectively according to relationship lifecycle requirements.
- Branch inventory uses a unique product/branch mapping to prevent duplicate stock records.
- Order items reference their parent order and purchased product.

- Bundle items associate multiple products with a single composite bundle.
- Enum-style status values constrain major workflow states.

4.6 Normalization

The source design documents normalization from an unnormalized spreadsheet-like structure through First Normal Form, Second Normal Form and Third Normal Form/BCNF. Repeating groups such as specifications, images and branch stock are separated into dedicated relations. Branch attributes are removed from composite inventory records, while category and brand information are separated to avoid transitive dependencies.

Stage	Key transformation
UNF	Product records may contain repeated specifications, images and branch stock information.
1NF	Multi-valued attributes are decomposed and values become atomic.
2NF	Partial dependencies are removed from composite product/branch stock relationships.
3NF / BCNF	Category and brand attributes are separated so non-key attributes do not depend transitively on product identifiers.

4.7 Indexing and Query Optimization

The documented schema includes indexes for frequently accessed product attributes such as SKU, category and price. The implementation also uses custom JPQL filtering, pagination and fetch joins. Fetch joins are used where appropriate to reduce repeated database access when retrieving product details together with related images and branch inventory.

5. RESTful API Design and Implementation

5.1 REST Principles

The API models business concepts as resources and uses standard HTTP methods. GET requests retrieve resources, POST requests create resources or initiate operations, PUT supports complete replacement, PATCH supports partial updates and DELETE handles removal or soft deletion where applicable.

Method	Typical use
GET	Retrieve resources or collections
POST	Create resources or execute creation-oriented operations
PUT	Replace an existing resource
PATCH	Partially modify a resource
DELETE	Delete or deactivate a resource

5.2 HTTP Status Codes

Status	Meaning
200	Successful request
201	Resource successfully created
204	Successful request with no response body
400	Invalid request or validation/business rule failure
401	Missing or invalid authentication
403	Authenticated but insufficient authorization
404	Resource not found
409	Conflict such as duplicate unique data
500	Unexpected server error

5.3 Response Envelope

The API uses a common response structure with status, message and body fields. This creates a consistent contract for the frontend and makes successful and unsuccessful responses easier to process.

Example: { "status": 200, "message": "Operation completed successfully", "body": { ... } }

5.4 API Module Catalogue

Controller	Base Path	Responsibility
AuthController	/api/v1/auth	Authentication and customer registration

UserController	/api/v1/users	User and staff administration
ProductController	/api/v1/products	Product CRUD, filtering and stock operations
CategoryController	/api/v1/categories	Category hierarchy and management
BrandController	/api/v1/brands	Brand management
BadgeController	/api/v1/badges	Badge management and automation
BranchController	/api/v1/branches	Branch information and nearest branch calculation
PromotionController	/api/v1/promotions	Deals, banners and bundles
OrderController	/api/v1/orders	Order placement and fulfilment
WishlistController	/api/v1/wishlist	Customer wishlist
ReviewController	/api/v1/reviews	Product reviews
TransferController	/api/v1/transfers	Inter-branch transfers
InventoryController	/api/v1/inventory	Inventory health and adjustments
NewsletterController	/api/v1/newsletter	Subscription and campaigns
BusinessProfileController	/api/v1/business-profile	Corporate profile
PolicyController	/api/v1/policies	Legal policies
ChatbotController	/api/v1/chat	AI technical advisor
AnalyticsController	/api/v1/analytics	Revenue, branch and product analytics
TestController	/api/v1/test	Health check

The source report documents 119 endpoints across the implemented controller modules. The endpoint count and module structure should be verified against the final repository before submission if code has changed since the report was generated.

5.5 Advanced Query Engineering

The project demonstrates optional filtering across category, brand, search text, price range and badge. It also uses hierarchical category joins, fetch joins for related collections, aggregation functions such as SUM and COUNT, and pageable results. These queries demonstrate the use of the persistence layer for both operational and reporting requirements.

5.6 Error Handling and Validation

Jakarta Bean Validation is applied to request DTOs. Validation failures are intercepted by the global exception handler and returned as structured field-level errors. This prevents malformed data from entering the business layer and gives the frontend actionable feedback.

6. Security, Authentication and Authorization

6.1 Authentication

The authentication architecture is stateless. After credential verification, a signed JWT is issued. The documented implementation uses HMAC SHA-512 signing and includes identity and authority information together with issued and expiration timestamps.

6.2 Password Protection

User passwords are not stored as plaintext. The implementation uses BCryptPasswordEncoder with a configured work factor of 12. This protects stored password representations against direct disclosure and increases the computational cost of password guessing attacks.

6.3 JWT Request Processing

17. The client submits credentials to the authentication endpoint.
18. The authentication provider validates the credentials.
19. A signed JWT is generated with the appropriate authority claims.
20. The client includes the token as a Bearer token on protected requests.
21. The JWT filter extracts and validates the token.
22. The authenticated principal and authorities are placed in the security context.
23. Controller-level or method-level authorization rules determine whether access is allowed.

6.4 Role-Based Access Control

Capability	Guest	Customer	Staff	Admin	SuperAdmin
Browse catalog	✓	✓	✓	✓	✓
Place order	✓	✓	✓	✓	✓
Wishlist	—	✓	—	—	—
Update fulfilment	—	—	✓	✓	✓
Inventory adjustments	—	—	✓	✓	✓
Transfer approval	—	—	—	✓	✓
User administration	—	—	—	✓	✓
Financial analytics	—	—	—	✓	✓
Master administration	—	—	—	—	✓

6.5 Request Validation

Validation constraints include required fields, string length limits, email format checks and password length requirements. The global exception handler converts validation failures into a consistent response structure rather than allowing framework-level errors to leak directly to clients.

6.6 CORS and Stateless Sessions

The backend configures Cross-Origin Resource Sharing so that the SPA can communicate with the API from its development origin. The security configuration uses stateless session management because authentication state is represented by bearer tokens rather than server-side HTTP sessions.

6.7 Security Considerations for Deployment

- Production credentials and JWT signing secrets should be supplied through environment variables or a secure secrets mechanism.
- HTTPS should be used for deployed environments.
- Development credentials shown in examples must never be reused in production.
- Access to administrative endpoints should be restricted by both authentication and authorization.
- Sensitive logs should avoid recording passwords, raw tokens or unnecessary personal information.
- Security testing should be repeated after major dependency or infrastructure changes.

7. Advanced Features and Value-Added Development

7.1 AI Technical Advisor

The system includes an AI-oriented technical advisor endpoint. The documented implementation classifies common customer intents such as product recommendations, shipping, warranty and return questions. It can inspect catalog information and return product identifiers that the frontend can render as recommendation cards.

7.2 Business Analytics

The analytics module provides executive metrics such as gross revenue, valid order counts, average order value, branch revenue contribution and top-selling products. These are generated using server-side aggregation queries.

7.3 Haversine Fulfilment Routing

The fulfilment engine compares customer coordinates with configured branch coordinates using the Haversine formula. The nearest active hub is selected and the system can calculate a distance-based shipping charge using the configured base rate and per-kilometre rate.

Haversine distance: $d = 2r \times \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos(\phi_1)\cos(\phi_2)\sin^2(\Delta\lambda/2))})$

7.4 Composite Bundle Availability

A deal bundle may require several component products. The maximum number of bundles that can be supplied is constrained by the component with the smallest relative availability. This prevents a bundle from being presented as available when one required component is exhausted.

Available Bundles = min over components of floor(Global Stock / Required Quantity)

7.5 Automated Product Badges

The badge rules engine uses configurable rules related to sales activity, inventory and views. Example outcomes include Hot or Trending labels for fast-moving products, Sale or Clearance labels for qualifying stock conditions and Limited Stock labels for very low inventory.

7.6 Newsletter Management

The newsletter module supports subscriber lifecycle management and administrative campaign composition. Campaign records can maintain subject information, HTML content, recipient counts, delivery counts and open-rate information.

7.7 Multi-Stage Stock Transfers

The transfer workflow models Draft, Pending, Approved, Rejected, Dispatched, Completed and Cancelled states. This creates an auditable operational sequence instead of allowing unrestricted inventory changes.

7.8 Docker Deployment

The backend contains a multi-stage Docker build using Java 21 runtime images. Environment variables can provide database connection details, port configuration and JWT secret configuration. The container is designed to run using an unprivileged application user.

8. Frontend SPA Integration

8.1 Frontend Architecture

The frontend is implemented as a lightweight Single Page Application using HTML, Vanilla ES6+ JavaScript and Tailwind CSS. Hash-based routing allows public and protected views to be switched without a traditional server-side routing framework.

8.2 Route Protection

Route Group	Access Rule
Public storefront	Accessible without authentication
Customer account	Requires authenticated customer session
Checkout	Requires an authenticated or supported ordering context
Administration	Requires staff/admin authorization
Unauthorized protected route	Redirect or access-denied handling

8.3 API Client

A dedicated JavaScript API client centralizes HTTP communication. The client reads the current JWT token and attaches it to protected requests. Responses are parsed into the common API body structure and HTTP errors are converted into frontend errors.

8.4 Local Persistence Fallback

The source implementation documents a dual-mode persistence strategy in which selected client data can fall back to localStorage if the backend is temporarily unreachable. This is useful during local development and limited offline testing, although authoritative business data remains server-side.

8.5 UI and UX

The documented visual language uses a light precision theme with engineering blue as the primary brand color, dark navy for high-contrast areas and cyan as a technical accent. Translucent cards, responsive layouts and custom modal notifications are used to provide a modern storefront and administration experience.

8.6 Frontend Modules

Module	Responsibilities
Authentication UI	Login, registration and session handling
Catalog UI	Product browsing, search and filtering
Product details	Specifications, features, images and reviews

Cart / checkout	Order preparation and submission
Account	Orders, wishlist and profile functions
Admin dashboard	Operational management and analytics
Inventory	Stock health, adjustments and transfers
Promotions	Deals, banners and bundles
Communication	Newsletter subscription and campaigns
AI advisor	Conversational hardware assistance

9. Testing and Quality Assurance

9.1 Testing Strategy

Testing is organized at multiple levels. Unit tests focus on individual services and utility logic. Integration tests validate Spring components and security behavior. API-level tests validate endpoint contracts and business workflows using tools such as Postman and cURL.

9.2 Test Execution Matrix

ID	Subsystem	Scenario	Expected Result	Status
TC-01	Authentication	Valid login	HTTP 200 and JWT returned	PASS
TC-02	Authentication	Invalid password	HTTP 401 Unauthorized	PASS
TC-03	Registration	Duplicate email	Validation/ conflict response	PASS
TC-04	Catalog	Category and price filter	Paginated matching results	PASS
TC-05	Catalog	Missing SKU	HTTP 400 validation error	PASS
TC-06	Logistics	Nearest branch for Galle coordinates	Galle branch selected	PASS
TC-07	Logistics	Nearest branch for Kandy coordinates	Kandy branch selected	PASS
TC-08	Orders	Valid stock order	Order created and stock changed	PASS
TC-09	Orders	Excessive quantity	Insufficient-stock rejection	PASS
TC-10	Transfers	Staff attempts approval	HTTP 403 Forbidden	PASS
TC-11	Transfers	Admin completes transfer	Source/ destination stock updated	PASS
TC-12	AI	Gaming laptop recommendation	Guidance and product IDs returned	PASS
TC-13	Analytics	Branch revenue request	Revenue breakdown returned	PASS

9.3 Security Verification

- Unauthenticated requests to protected user endpoints are rejected.
- Customer accounts cannot access administrator analytics and management functions.
- Standard administrators cannot perform SuperAdmin-only deletion operations.
- Staff users cannot approve stock transfers where authorization rules restrict that action.

9.4 Data Integrity Verification

Order placement validates stock before applying the inventory mutation. Stock transfer completion changes both source and destination inventory inside a transaction. Foreign keys and unique constraints provide database-level integrity protection.

9.5 Performance and Query Quality

The design uses pagination, indexed fields and fetch joins to reduce unnecessary database round trips. The original specification defines a target of efficient standard read queries under normal operational load; actual performance measurements should be reported separately if formal load-testing results are available.

9.6 Logging and Version Control

The source implementation uses SLF4J/Logback through Lombok logging annotations and documents meaningful Git commit conventions such as feat, fix, refactor and docs. Logging supports troubleshooting of API requests, validation failures and operational events.

10. Deployment, Maintenance and Future Work

10.1 Development Environment

Requirement	Documented configuration
JDK	Java 21 / Eclipse Temurin recommended
Database	MySQL 8.x
Build tool	Apache Maven 3.9+
Backend	Spring Boot application on port 8080
Frontend	Browser or local HTTP server on port 5500
Container	Docker multi-stage image

10.2 Deployment Procedure

24. Install Java 21, MySQL 8.x and Maven.
25. Create the ETech database schema and configure application properties.
26. Start the Spring Boot backend.
27. Start or serve the SPA frontend.
28. Verify authentication and database connectivity.
29. For cloud deployment, build the Docker image and provide database/JWT settings through environment variables.
30. Run functional and security checks before exposing the system to users.

10.3 Maintainability Strategy

- Keep controller, service, repository and entity responsibilities separated.
- Use DTOs to maintain stable API contracts.
- Centralize validation and exception handling.
- Use database constraints to protect data integrity.
- Keep configuration externalized for deployment.
- Use version control with meaningful commits.
- Maintain test coverage around security and inventory mutations.

10.4 Technical Challenges and Solutions

Challenge	Solution
N+1 database queries	Custom fetch joins for product-related data
Bundle overselling	Real-time component bottleneck calculation
CORS restrictions	Explicit CORS configuration in Spring Security
Multi-branch shipping inconsistency	Coordinate-based Haversine routing
Complex stock workflows	Transactional multi-stage transfer lifecycle

10.5 Future Enhancement Roadmap

Enhancement	Purpose
Payment integration	Add payment gateway webhooks and secure asynchronous payment confirmation.
SMS notifications	Provide dispatch and delivery notifications through an SMS gateway.
Advanced search	Introduce Elasticsearch for autocomplete, typo tolerance and richer search.
Mobile application	Create Flutter or React Native clients using the existing REST API.
Audit expansion	Introduce a dedicated immutable audit-log subsystem for high-value administrative events.
Operational monitoring	Add centralized metrics, tracing and alerting for production environments.

10.6 Conclusion

The ETech Computers platform demonstrates a complete backend-oriented software engineering solution for a multi-branch computer hardware business. It brings together REST API design, authentication, authorization, relational database engineering, frontend integration and advanced business functions in one coherent architecture.

The implementation addresses the central requirements of the ITS1114 Advanced API Development module by demonstrating resource-oriented REST endpoints, secure authentication and authorization, a normalized MySQL database and advanced SQL/JPQL operations. The additional AI advisory, analytics, routing, bundle, marketing and transfer features provide further evidence of applied engineering beyond the minimum CRUD requirements.

The platform also provides a practical foundation for future expansion. Because the business logic is exposed through a REST API, additional clients such as mobile applications, POS systems or warehouse interfaces can be developed without replacing the core backend architecture.

References

31. Walls, C. (2022). Spring in Action. 6th ed. Manning Publications.
32. Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
33. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
34. Hibernate Community. (2024). Hibernate ORM User Guide.
35. Spring Projects. (2024). Spring Boot Reference Documentation.
36. JSON Web Token Working Group. (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF).

Appendix A — Database Design Summary

The implemented database contains 24 tables covering branches, users, catalog taxonomy, products, product assets, inventory, orders, customer engagement, transfers, promotions, bundles, newsletters, corporate information and legal policies. The full DDL is maintained as part of the project source and may be attached as a separate appendix file where the academic submission requires complete SQL.

Appendix B — Representative API Payloads

B.1 Login

POST /api/v1/auth/login

Request: { "username": "<username>", "password": "<password>" }

Response: { "status": 200, "message": "Login successful", "body": { "token": "<JWT>", "username": "<username>", "role": "<role>" } }

B.2 Product Filtering

GET /api/v1/products/filter?

category=laptops&minPrice=200000&maxPrice=500000&page=0&size=10

B.3 Order Placement

POST /api/v1/orders

Request contains customer information, shipping information, fulfilment branch and an array of product IDs with quantities.

B.4 AI Advisor

POST /api/v1/chat/message

Request: { "message": "Recommend a high-end gaming laptop." }

Appendix C — Deployment Configuration

Local development requires JDK 21, MySQL 8.x and Maven. The backend is configured for port 8080 and the frontend can be served through a local HTTP server. Cloud deployment should inject database credentials, JWT signing secrets and other environment-specific settings rather than storing secrets in source code.

Appendix D — Academic Submission Checklist

- Confirm student name and registration number on the cover page.
- Confirm lecturer/supervisor details if required by IJSE.
- Update the automatic Table of Contents after final pagination.
- Insert final screenshots of the implemented frontend and admin console.
- Insert the final ERD generated from the actual database schema.
- Attach the complete source code/repository reference according to submission rules.
- Verify all API counts and feature claims against the final repository before submission.
- Remove all real passwords, API keys, JWT secrets and production credentials from the submitted report.
- Run final spelling, page-number and cross-reference checks.

Appendix E — Detailed Module Descriptions

Module	Description
Authentication	Provides registration, login, token verification and authenticated user context.
User Management	Maintains staff/customer identities, account statuses, roles and branch assignments.
Catalog	Maintains products and associated brands, categories, badges, images, specifications and features.
Inventory	Maintains branch-specific stock and supports low-stock monitoring.
Transfers	Controls inter-branch inventory movement through approval and completion states.
Orders	Handles customer order creation, line items, stock validation, financial totals and tracking.
Reviews	Captures customer feedback and product ratings.
Wishlist	Stores customer-selected products and

	supports batch cart movement.
Promotions	Provides hot deals, home banners and bundle structures.
Newsletter	Maintains subscribers and administrative campaign records.
Analytics	Aggregates sales and branch metrics for management views.
Chatbot	Provides intent-oriented hardware assistance and catalog recommendations.
Policies	Stores structured customer-facing policy content.
Business Profile	Stores public enterprise contact and profile configuration.

Appendix F — Design Review Notes

The final implementation should be treated as the authoritative source for exact controller counts, dependency versions, deployment commands and endpoint totals. This report intentionally presents the documented project architecture and features in a submission-ready academic format while avoiding unsupported claims about measurements or production operation.

Where the original implementation uses example credentials or placeholder cloud values, those values are represented generically in this report. Academic screenshots and API demonstrations should use non-sensitive demonstration accounts and secrets.

Detailed Implementation: Authentication and User Management

Purpose

The authentication subsystem establishes the identity and access boundary for the platform. It supports public registration, credential verification, signed token issuance and authenticated request processing.

Registration

Customer registration accepts validated identity information and creates a CUSTOMER account. Username and email uniqueness are enforced at the persistence level.

Login

The login endpoint authenticates supplied credentials and returns a JWT containing the authenticated identity and authority claims.

Token processing

The JWT filter extracts the bearer token, validates its signature and expiry, loads user details and places the resulting authentication object into the Spring Security context.

Administration

Administrative users can manage staff/customer accounts, role assignments and account statuses subject to authorization rules.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Product Catalog

Catalog model

Products are separated from categories, brands, badges, images, specifications and features so that the catalog can evolve without duplicating descriptive data.

SKU management

Each product has a unique SKU. The SKU is used as an operational identifier for catalogue lookup and inventory-related activities.

Technical data

The specs relation supports key-value technical attributes while features stores presentation-oriented bullet points.

Gallery

Product images are stored independently with a display order, allowing the frontend to render a controlled gallery.

Filtering

The API supports category, brand, search text, price range and badge filtering together with pagination.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Multi-Branch Inventory

Inventory model

Branch inventory is represented by a product/branch relationship with a stock quantity. A unique constraint prevents duplicate mappings.

Global availability

The application can aggregate branch quantities to determine whether a product is available somewhere in the network.

Low-stock control

Products can define a low-stock margin and an alert flag. This allows staff to focus on inventory that requires attention.

Operational value

Centralized branch inventory reduces the need to contact individual branches simply to establish current availability.

Integrity

Inventory mutations are performed within service-layer transactions where the operation requires atomicity.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Order Processing

Order creation

An order contains customer, delivery, fulfilment and financial information while order_items represents the purchased products.

Stock validation

Before an order is committed, the selected branch inventory is checked against every requested quantity.

Financial calculation

The service calculates subtotal, tax, shipping and grand total and persists the resulting order.

Tracking

A unique human-readable order code provides a public tracking reference.

Lifecycle

Orders progress through Pending, Processing, Shipped, Delivered or Cancelled states.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Stock Transfer Workflow

Request

Staff create a transfer specifying source branch, destination branch, product and quantity.

Validation

The service checks whether the source branch contains sufficient stock.

Approval

The request follows a controlled status sequence and administrator approval is required for approval-oriented transitions.

Completion

At completion, source stock is reduced and destination stock is increased as one transactional operation.

Auditability

The transfer record retains initiator, approver, status and timestamps, creating an operational history.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Promotions and Bundles

Hot deals

A hot deal associates a product with a promotional price, discount percentage and active period.

Home banners

Homepage promotional content is stored separately so marketing presentation does not need to modify the core product entity.

Bundles

Deal bundles represent a composite offer containing several product components.

Availability algorithm

Bundle availability is determined by the minimum relative availability of all required components.

Overselling protection

A bundle cannot be considered deliverable beyond the availability of its bottleneck component.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Analytics

Overview

The analytics layer aggregates gross revenue, valid order counts, average order value and active user metrics.

Branch analysis

Branch revenue is grouped by fulfilment hub to provide management with regional contribution information.

Product ranking

Order-item history is aggregated to identify high-volume and high-revenue products.

Query design

Aggregation is performed in the data-access layer so large calculations do not need to be assembled inefficiently in the browser.

Use in management

The results support operational monitoring and business decision-making rather than replacing formal accounting systems.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: AI Technical Advisor

Purpose

The advisor provides conversational assistance for common hardware and store-related questions.

Intent handling

Documented intents include shipping, warranty, returns and product recommendation requests.

Catalog integration

Recommendation responses can inspect live product information and return product IDs for frontend rendering.

Frontend experience

Suggested products can be displayed as interactive product cards rather than plain text alone.

Boundary

The advisor is treated as a technical assistance feature; authoritative business information remains controlled by the application's stored data and policies.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Logistics Engine

Coordinates

The branch model stores latitude and longitude values for regional hubs.

Distance

The Haversine calculation estimates great-circle distance between a destination coordinate and each active branch.

Selection

The branch with the lowest calculated distance can be selected as the preferred fulfilment hub.

Shipping

The documented calculation combines a base branch rate with a per-kilometre component.

Extensibility

The same service can later be connected to richer routing or delivery-provider APIs without changing the customer-facing order resource.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Frontend Integration

SPA model

The frontend uses a browser-based Single Page Application structure with hash-based navigation.

API client

HTTP communication is centralized through JavaScript API modules.

Authentication

The API client can attach the active JWT bearer token to requests that require authentication.

Route guards

Protected customer and administration views perform authentication/role checks before rendering restricted content.

Fallback

The documented client includes localStorage-based fallback persistence for selected data during backend unavailability.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Error Handling

Validation

Jakarta validation annotations reject malformed or incomplete request DTOs before business logic executes.

Global handler

A centralized exception handler maps validation and operational exceptions into predictable response structures.

Client contract

The common response envelope provides status, message and body fields.

HTTP semantics

The API distinguishes validation errors, authentication failures, authorization failures, missing resources and conflicts using appropriate status codes.

Maintainability

Centralized error mapping prevents every controller from implementing its own inconsistent error format.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Database Integrity

Primary keys

Every entity has a stable primary key appropriate to its role.

Foreign keys

Relationships between products, categories, branches, orders, users and other entities are enforced through foreign keys.

Unique constraints

Unique values are used for fields such as SKU, username, email and selected business identifiers.

Cascades

Cascade rules are applied selectively to dependent records such as product images and order items.

Normalization

The schema separates independent concepts to reduce duplication and update anomalies.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Testing Evidence

Authentication tests

Valid credentials, invalid credentials and duplicate registration conditions are included in the documented test matrix.

Catalog tests

Product filtering and missing-SKU validation are tested.

Inventory tests

Valid stock orders and insufficient-stock conditions are tested.

Security tests

Staff authorization boundaries and administrator permissions are tested.

Advanced tests

Nearest-branch routing, stock transfer completion, AI recommendation and analytics retrieval are included.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Detailed Implementation: Deployment

Local prerequisites

The documented environment uses JDK 21, MySQL 8.x and Maven 3.9+.

Configuration

Database and server settings are supplied through application properties or environment variables.

Frontend serving

The SPA can be opened directly or served using a local HTTP server.

Container build

A multi-stage Docker build separates compilation from the production runtime image.

Cloud readiness

Database connection details, port configuration and JWT secrets are parameterized for deployment environments.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.

Professional Review and Submission Guidance

Accuracy

Before final submission, endpoint totals, dependency versions and feature claims should be checked against the final repository.

Evidence

Screenshots of the final storefront, admin console, API testing and database should be inserted where required by the academic marking scheme.

Security

Real credentials, JWT secrets, API keys and production database passwords must never appear in the report.

Academic tone

The report describes implemented behavior without making unsupported claims about measured production performance.

Reproducibility

The appendix and deployment instructions should allow an evaluator to understand how the system is configured and executed.

Implementation Evidence Checklist

- Requirement is represented in the documented architecture and implementation.
- Business logic is separated from HTTP transport and persistence concerns.
- Validation and access control are applied at appropriate boundaries.
- The feature is testable through a defined request/response or workflow.